

DATABASE EXECUTION PLAN

Dawwarha — MongoDB Implementation Plan

4-Engineer Database Execution Plan · MEAN Stack ·
NTI Graduation Project

VERSION

1.0

SOURCE

Derived from Dawwarha — Product Brief /
Concept Report v1.0

COLLECTIONS

10 domain-owned MongoDB collections

DOCUMENT STATUS

Single source of truth for the database layer

IMPLEMENTATION TEAM

رحمة محمد محمد أحمد
دينا عبدالله عليان محمد

ادهم محمد حسن عبد السميع
عبدالستار محمد عبدالستار مطاوع

Database plan at a glance

A structured execution plan for the data layer behind Dawwarha's
Resource → Request → Match → Handover → Impact loop.

01	Executive Database Architecture	3
02	Database Goals	4
03	Product-to-Database Traceability	5
04	Domain Map	6
05	Complete Collection Inventory	7
06	Canonical Data Model	8
07	Embedding vs. Referencing Decisions	15
08	Schema Specifications (Mongoose)	17
09	Enum & Lifecycle Design	21
10	Index Strategy	23
11	Validation Strategy	25
12	Data Integrity Strategy	26
13	Security Strategy	28
14	Aggregation Strategy	29
15	Transaction Strategy	30
16	Seed Data Strategy	31
17	API-to-Database Mapping	32
18	Shared Database Contract	34
19	Team Architecture	36
20	Engineer 1 Plan — رحمة محمد محمد أحمد	37
21	Engineer 2 Plan — ادهم محمد حسن عبد السميع	40
22	Engineer 3 Plan — دينا عبدالله عليان محمد	42
23	Engineer 4 Plan — عبدالستار محمد عبدالستار مطاوع	44
24	Git & Collaboration Strategy	47
25	Integration Plan	48
26	Implementation Order	49
27	Database Test Plan	50
28	End-to-End Database Scenario	52
29	Definition of Done	54
30	Student Execution Checklists	55
31	Final Architecture Summary	57

01 — Executive Database Architecture

Dawwarha’s database layer exists to make one promise reliable: **a resource offered by one person can be found by, matched to, and safely handed over to the person who needs it — with a verifiable record at the end.** Everything in this plan traces back to that loop: **Resource → Request → Match → Handover → Impact**, wrapped by **Identity & Access** (who is allowed to do what) and **Trust & Safety** (what happens when something goes wrong).

The architecture uses **10 collections**, each with exactly one owning engineer. Relationships are expressed mostly through **references (ObjectId)**, consistent with the Product Brief’s explicit direction in Section 18/19 (“relationships expressed via references rather than deep embedding”). A small number of genuinely small, immutable, or always-read-together fields are embedded — never as a blanket strategy, but decided individually in Section 07.

The plan is organized as four domain-owned work packages that mirror the product’s own structure, not an arbitrary split:

Domain	Engineer	Collections Owned
Identity & Access	رحمة محمد محمد أحمد	users, organizations
Supply	ادهم محمد حسن عبد السميع	resources, categories
Demand & Matching	دينا عبدالله عليان محمد	requests, matches
Transfer, Trust & Impact	عبدالستار محمد عبدالستار مطاوع	handovers, reports, notifications, contributions

No collection is owned by two engineers. Any engineer may **read** another domain’s collection (via a reference ID) but may not add fields to it or change its schema without going through the Shared Database Contract (Section 18) and a cross-domain PR review.

02 — Database Goals

1. **Traceability** — every collection, field, and index must map to a Functional Requirement (FR-xxx) or Non-Functional Requirement (NFR-xxx) from the Product Brief, or to a named MVP feature in Section 10/31 of the brief.
2. **Explainability** — the matching engine is rule-based (Section 12 of the brief). The database must store *why* a match scored what it scored, not just the number.
3. **Accountability** — a transfer only becomes real when both sides confirm it (Section 08, FR-013). The schema must make one-sided completion structurally impossible, not just discouraged.
4. **Ownership clarity** — a user must never be able to modify another user's data at the database-authorization boundary (`ownerId` / `providerId` / `requesterId` fields exist specifically to support this).
5. **Beginner-safe execution** — every task in Sections 20–23 is written so a first-time MongoDB/Mongoose user can complete it without guessing.
6. **No over-engineering** — no Kafka, no Redis, no Elasticsearch, no vector search, no ML infrastructure, no premature sharding. MongoDB + Mongoose + well-chosen indexes is sufficient for this MVP's scale (a graduation demo and early pilot use, not production traffic).

03 — Product-to-Database Traceability

Product Brief Requirement	Database Consequence
FR-001–FR-003 (register/login/logout)	<code>users</code> collection, hashed <code>passwordHash</code> , unique <code>email</code> index
FR-004 (RBAC for User/Organization/Admin)	<code>users.role</code> enum, <code>organizations</code> verification-gated features
FR-005–FR-006 (Resource CRUD)	<code>resources</code> collection, <code>providerId</code> ownership field
FR-007–FR-008 (Request CRUD)	<code>requests</code> collection, <code>requesterId</code> ownership field
FR-009–FR-010 (match scoring + ranked candidates)	<code>matches</code> collection with explainable <code>scoreBreakdown</code>
FR-011 (accept/reject match)	<code>matches.status</code> enum + transition rules
FR-012 (resource lifecycle)	<code>resources.status</code> enum + State Transition Matrix (Section 09)
FR-013 (two-sided confirmation)	<code>handovers.confirmedByProvider</code> / <code>confirmedBySeeker</code>
FR-014–FR-015 (reporting + admin review)	<code>reports</code> collection, polymorphic <code>targetType</code> / <code>targetId</code>
FR-016–FR-017 (org verification)	<code>organizations.verification.status</code> lifecycle
FR-018 (contribution history)	<code>contributions</code> collection + derived counters on <code>users</code> / <code>organizations</code>
FR-019 (admin dashboard)	Aggregation pipelines in Section 14, no separate “analytics” collection
FR-022 (search/filter by category/location)	Compound indexes on <code>resources</code> / <code>requests</code>
FR-024 (controlled category taxonomy, admin-only)	<code>categories</code> collection, admin-only write
NFR-002 (password hashing)	bcrypt hash stored, plaintext never persisted
NFR-009 (backend-enforced schema validation)	Mongoose schema validation on every write path

No collection in this plan exists without a row in this table or a clearly labeled architectural necessity (e.g. index-support fields).

```
graph TD; IA[Identity & Access] --- S[Supply]; S -.-> D[Demand]; D --- TS[Trust & Safety]; TS --- M[Matching]; M --- T[Transfer]; T -.-> IA; S -.-> M; M -.-> T; S -.-> L((Dawwarha loop)); M -.-> L; L -.-> D; L -.-> T; L -.-> IA; L -.-> S;
```

The diagram illustrates the Dawwarha loop, a central concept connecting six main components arranged in a circular flow:

- Identity & Access** (Dark Blue):
 - users
 - organizations
- Supply** (Teal):
 - resources
 - categories
- Demand** (Orange):
 - requests
- Trust & Safety** (Orange):
 - reports
- Matching** (Teal):
 - matches
- Transfer** (Dark Blue):
 - handovers

The flow is as follows:

- Identity & Access** connects to **Supply** via a solid red line.
- Supply** connects to **Demand** via a dashed grey line.
- Demand** connects to **Trust & Safety** via a solid red line.
- Trust & Safety** connects to **Matching** via a solid red line.
- Matching** connects to **Transfer** via a solid red line.
- Transfer** connects back to **Identity & Access** via a solid red line.

Additionally, a central **Dawwarha loop** (dark blue oval) is connected to all six components via dashed grey lines, indicating its central role in the process.

This differs slightly from the generic template in one deliberate way: **categories sits under Supply, not as a standalone domain**. The Product Brief treats categories as controlled taxonomy consumed by both Resources and Requests, but it is created and shaped primarily to describe *what can be offered* — and giving it its own engineer would create a near-empty work package. Engineer 2 (Supply) owns it; Engineer 3 (Demand & Matching) consumes it read-only.

05 — Complete Collection Inventory

#	Collection	Purpose	Owner	Consumers	Scope
1	users	Individual accounts (providers, seekers, admins)	Engineer 1	All engineers	MVP
2	organizations	Verified entities acting at scale	Engineer 1	Engineer 2, 3, 4	MVP
3	resources	Surplus listings	Engineer 2	Engineer 3, 4	MVP
4	categories	Controlled taxonomy	Engineer 2	Engineer 1 (n/a), 3	MVP
5	requests	Published needs	Engineer 3	Engineer 2 (n/a), 4	MVP
6	matches	Scored resource↔request candidates	Engineer 3	Engineer 4	MVP
7	handovers	Two-sided transfer confirmation	Engineer 4	—	MVP
8	reports	Moderation flags (polymorphic target)	Engineer 4	—	MVP
9	notifications	User-facing alerts	Engineer 4	—	MVP (schema now; P1 delivery)
10	contributions	Impact/history ledger events	Engineer 4	—	MVP (minimal)

Every collection has exactly one primary owner. No collection is left unassigned.

06 — Canonical Data Model

For each collection: purpose, owner, consumers, fields, defaults, enums, references, embeds, validation, indexes, lifecycle, and key queries.

6.1 users — Owner: Engineer 1

Purpose: Represents every individual account — providers and seekers are the same role (`user`) distinguished only by *behavior* (what they publish), per the Product Brief’s persona model. Admins are a distinct role.

Field	Type	Required	Default	Notes
<code>_id</code>	ObjectId	generated	—	
<code>name</code>	String	✓	—	trimmed, 2–80 chars
<code>email</code>	String	✓	—	unique, lowercase, indexed
<code>passwordHash</code>	String	✓	—	bcrypt hash only, never plaintext
<code>role</code>	String enum	✓	"user"	<code>user</code> , <code>admin</code> (organization is a separate collection, not a user role — see 6.2)
<code>location</code>	Object { <code>city</code> , <code>area</code> }	optional	—	free-text MVP location (see Section 11 for geospatial decision)
<code>status</code>	String enum	✓	"active"	<code>active</code> , <code>suspended</code>
<code>emailVerifiedAt</code>	Date	optional	null	set on verification (FR basic email verification)
<code>reputationScore</code>	Number	✓	0	derived/cached — see Section 6.10 reasoning
<code>stats.completedTransfers</code>	Number	✓	0	cached counter, source of truth is <code>contributions</code>
<code>stats.failedTransfers</code>	Number	✓	0	cached counter
<code>createdAt</code> / <code>updatedAt</code>	Date	auto	—	<code>timestamps: true</code>

Validation: `email` matches email format + uniqueness; `passwordHash` never accepted directly from client input (hashing happens in the auth service, not the schema); `role` restricted to enum; suspended users blocked at the authorization layer, not deleted.

Indexes:

- { `email`: 1 } unique — login lookup, duplicate-registration prevention (FR-001, NFR)
- { `status`: 1 } — admin “manage accounts” queries

Lifecycle: `active` → `suspended` (admin action) → `active` (admin reactivation). No hard delete — see Section 12.5.

6.2 organizations — Owner: Engineer 1

Purpose: A verified entity (NGO/school/community center) that can publish standing needs at scale once approved.

Field	Type	Required	Default	Notes
<code>_id</code>	ObjectId	generated	—	
<code>ownerUserId</code>	ObjectId → <code>users</code>	✓	—	the individual who registered/manages the org
<code>name</code>	String	✓	—	
<code>description</code>	String	optional	—	max 1000 chars
<code>contactInfo</code>	Object { <code>phone</code> , <code>email</code> , <code>address</code> }	optional	—	shared only after match acceptance per Product Brief privacy note
<code>verification.status</code>	String enum	✓	"pending"	<code>pending</code> , <code>approved</code> , <code>rejected</code> , <code>suspended</code>
<code>verification.submittedDocuments</code>	Array (URLs/refs)	optional	[]	bounded — see Section 13
<code>verification.reviewedBy</code>	ObjectId → <code>users</code> (admin)	optional	null	
<code>verification.reviewedAt</code>	Date	optional	null	
<code>verification.rejectionReason</code>	String	optional	null	required when status is <code>rejected</code>
<code>createdAt</code> / <code>updatedAt</code>	Date	auto	—	

Why `ownerUserId` and not “organization is a user role”: the Product Brief models Organization as its own entity with its own verification lifecycle, separate contact info, and separate CRUD surface (Section 15). Folding it into `users.role = "organization"` would force one physical person’s account to *become* the org, which breaks the “owner manages org, org has its own identity” relationship the brief describes (“Belongs to a User (owner)”).

Validation: `verification.status` transitions only via admin action (enforced in the service layer; the schema documents allowed values). `rejectionReason` required-if-rejected is enforced at the application layer (Mongoose custom validator).

Indexes:

- { `ownerUserId: 1` } — “my organization” lookup
- { `"verification.status": 1, createdAt: 1` } — admin’s pending-verification queue (FIFO)

Lifecycle: `pending` → `approved` | `rejected`; `approved` → `suspended` (admin, policy violation); `rejected` → `pending` (resubmission, P1 — MVP can require a new org record or a simple resubmit flag).

6.3 categories — Owner: Engineer 2

Purpose: Controlled taxonomy shared by `resources` and `requests`, admin-managed only (FR-024).

Field	Type	Required	Default	Notes
<code>_id</code>	ObjectId	generated	—	
<code>name</code>	String	✓	—	unique
<code>slug</code>	String	✓	—	unique, URL-safe, generated from <code>name</code>
<code>description</code>	String	optional	—	
<code>isActive</code>	Boolean	✓	true	soft-disable instead of delete (Section 12)
<code>createdAt</code> / <code>updatedAt</code>	Date	auto	—	

No `parentCategory` field — the brief lists a flat set (food, books, furniture, clothing, school supplies, office equipment, other). A hierarchical taxonomy is explicitly not justified by the MVP and is excluded per the “do not create unnecessary hierarchical taxonomy” restriction.

Indexes: `{ slug: 1 }` unique, `{ isActive: 1 }`.

Lifecycle: `isActive: true → false` (admin deactivates instead of deleting, to avoid breaking historical `resources` / `requests` that reference it).

6.4 resources — Owner: Engineer 2

Purpose: A surplus listing published by a provider (user or organization).

Field	Type	Required	Default	Notes
<code>_id</code>	ObjectId	generated	—	
<code>providerId</code>	ObjectId → <code>users</code>	✓	—	ownership field for authorization
<code>providerOrgId</code>	ObjectId → <code>organizations</code>	optional	<code>null</code>	set when an org publishes on its own behalf
<code>categoryId</code>	ObjectId → <code>categories</code>	✓	—	
<code>title</code>	String	✓	—	3–100 chars
<code>description</code>	String	✓	—	max 1000 chars
<code>quantity</code>	Number	✓	—	> 0
<code>location</code>	Object { <code>city</code> , <code>area</code> }	✓	—	
<code>availabilityWindow.start</code>	Date	✓	—	
<code>availabilityWindow.end</code>	Date	✓	—	must be after <code>start</code>
<code>status</code>	String enum	✓	<code>"draft"</code>	see Section 09 lifecycle
<code>safetyDisclosure</code>	String	conditional	<code>null</code>	required when category is a “sensitive” one (e.g. food)
<code>createdAt</code> / <code>updatedAt</code>	Date	auto	—	

Embedding decision: `location` and `availabilityWindow` are embedded (small, always read with the resource, never independently queried across resources). `providerId` / `categoryId` are references (see Section 07 for full reasoning).

Indexes:

- `{ status: 1, categoryId: 1, createdAt: -1 }` — “browse available resources by category, newest first” (FR-022)
- `{ providerId: 1, status: 1 }` — “my resources” list
- `{ "location.city": 1, "location.area": 1, status: 1 }` — location filter (FR-022)
- `{ "availabilityWindow.end": 1 }` — background job to auto-expire listings

Lifecycle: full state machine in Section 09.

6.5 requests — Owner: Engineer 3

Purpose: A published need from a seeker or organization.

Field	Type	Required	Default	Notes
<code>_id</code>	ObjectId	generated	—	
<code>requesterId</code>	ObjectId → <code>users</code>	✓	—	
<code>requesterOrgId</code>	ObjectId → <code>organizations</code>	optional	<code>null</code>	
<code>categoryId</code>	ObjectId → <code>categories</code>	✓	—	
<code>quantity</code>	Number	✓	—	> 0
<code>urgency</code>	String enum	✓	<code>"medium"</code>	<code>low</code> , <code>medium</code> , <code>high</code>
<code>location</code>	Object { <code>city</code> , <code>area</code> }	✓	—	
<code>description</code>	String	optional	—	max 500 chars
<code>status</code>	String enum	✓	<code>"draft"</code>	see Section 09
<code>createdAt</code> / <code>updatedAt</code>	Date	auto	—	

Indexes:

- { `status`: 1, `categoryId`: 1, `urgency`: -1, `createdAt`: -1 } — “open requests by urgency” (matching candidate retrieval + FR-022)
- { `requesterId`: 1, `status`: 1 } — “my requests”
- { `"location.city"`: 1, `"location.area"`: 1, `status`: 1 }

Lifecycle: `draft` → `published` → `matched` → `accepted` → `fulfilled`, plus `cancelled`, `expired` — full matrix in Section 09.

6.6 matches — Owner: Engineer 3

Purpose: A scored, explainable candidate connection between one `resource` and one `request`.

Field	Type	Required	Default	Notes
<code>_id</code>	ObjectId	generated	—	
<code>resourceId</code>	ObjectId → <code>resources</code>	✓	—	
<code>requestId</code>	ObjectId → <code>requests</code>	✓	—	
<code>providerId</code>	ObjectId → <code>users</code>	✓	—	denormalized copy for query speed (see 07.4)
<code>requesterId</code>	ObjectId → <code>users</code>	✓	—	denormalized copy
<code>score</code>	Number	✓	—	total weighted score, 0–1
<code>scoreBreakdown</code>	Object	✓	—	{ <code>category</code> , <code>location</code> , <code>quantity</code> , <code>urgency</code> , <code>availability</code> }, each 0–1
<code>status</code>	String enum	✓	<code>"proposed"</code>	<code>proposed</code> , <code>accepted</code> , <code>rejected</code> , <code>expired</code>
<code>expiresAt</code>	Date	optional	—	timeout for no-response matches (Product Brief “match fails... timeout”)
<code>createdAt</code> / <code>updatedAt</code>	Date	auto	—	

Why `providerId` / `requesterId` are denormalized here: every “my matches” query for a provider or requester needs to filter matches without joining through `resources` / `requests` first. This is a deliberate, justified duplication (see Section 07.4) — not a rule, an exception with a stated reason.

Indexes:

- `{ resourceId: 1, status: 1 }` — matches for a resource
- `{ requestId: 1, status: 1 }` — matches for a request
- `{ providerId: 1, status: 1, createdAt: -1 }` — provider’s match inbox
- `{ requesterId: 1, status: 1, createdAt: -1 }` — requester’s match inbox
- `{ status: 1, expiresAt: 1 }` — background job to expire stale matches

Uniqueness rule: a `(resourceId, requestId)` pair should not have two simultaneous `proposed` matches. Enforced via a partial unique index: `{ resourceId: 1, requestId: 1 }` unique, filtered to `status: "proposed"`.

6.7 handovers — Owner: Engineer 4

Purpose: Two-sided confirmation of a physical transfer, tied 1:1 to an accepted `match`.

Field	Type	Required	Default	Notes
<code>_id</code>	ObjectId	generated	—	
<code>matchId</code>	ObjectId → <code>matches</code>	✓	—	unique — one handover per match
<code>resourceId</code>	ObjectId → <code>resources</code>	✓	—	denormalized for query/reporting convenience
<code>requestId</code>	ObjectId → <code>requests</code>	✓	—	denormalized
<code>providerId</code>	ObjectId → <code>users</code>	✓	—	
<code>seekerId</code>	ObjectId → <code>users</code>	✓	—	
<code>confirmedByProvider</code>	Boolean	✓	<code>false</code>	
<code>confirmedBySeeker</code>	Boolean	✓	<code>false</code>	
<code>providerConfirmedAt</code>	Date	optional	<code>null</code>	
<code>seekerConfirmedAt</code>	Date	optional	<code>null</code>	
<code>status</code>	String enum	✓	<code>"in_progress"</code>	<code>in_progress</code> , <code>completed</code> , <code>cancelled</code> , <code>no_show</code>
<code>completedAt</code>	Date	optional	<code>null</code>	set only when both confirmations are true
<code>createdAt</code> / <code>updatedAt</code>	Date	auto	—	

Indexes:

- `{ matchId: 1 }` unique
- `{ providerId: 1, status: 1 }`, `{ seekerId: 1, status: 1 }` — “my pending handovers”

Lifecycle / state machine: full detail in Section 6.7.1 below and Section 09.

6.7.1 Two-sided confirmation logic

- Only `providerId` may set `confirmedByProvider`; only `seekerId` may set `confirmedBySeeker` — enforced in the service layer using the ownership fields.
- Setting a confirmation a second time is a no-op (idempotent), not an error — prevents duplicate-click issues.

- When **both** are `true`, the service sets `status: "completed"` and `completedAt: now()`, and triggers a `contributions` write (Engineer 4's own collection — no cross-domain write needed).
- `cancelled` and `no_show` are terminal states set by either party (cancel) or a timeout job (no-show), matching the Product Brief's "match fails... timeout... repeated failures visible to moderation."

6.8 reports — Owner: Engineer 4

Purpose: A flag on a listing, request, or user profile — polymorphic target.

Field	Type	Required	Default	Notes
<code>_id</code>	ObjectId	generated	—	
<code>reporterId</code>	ObjectId → <code>users</code>	✓	—	
<code>targetType</code>	String enum	✓	—	<code>resource</code> , <code>request</code> , <code>user</code>
<code>targetId</code>	ObjectId	✓	—	polymorphic — points into <code>resources</code> , <code>requests</code> , or <code>users</code> depending on <code>targetType</code>
<code>reason</code>	String enum	✓	—	<code>spam</code> , <code>fraud</code> , <code>inappropriate</code> , <code>safety</code> , <code>other</code>
<code>description</code>	String	optional	—	max 500 chars
<code>status</code>	String enum	✓	<code>"open"</code>	<code>open</code> , <code>reviewed</code> , <code>resolved</code>
<code>reviewedBy</code>	ObjectId → <code>users</code> (admin)	optional	<code>null</code>	
<code>resolution</code>	String	optional	<code>null</code>	
<code>createdAt</code> / <code>updatedAt</code>	Date	auto	—	

Polymorphic-reference strategy: `targetType` + `targetId` instead of three optional foreign-key fields (`resourceId`, `requestId`, `userId`, mostly `null`). This is the safe MongoDB pattern for “one of several possible parents”: the application layer, not the schema, is responsible for validating that `targetId` actually resolves against the collection named by `targetType` before saving (a Mongoose pre-save hook or service-layer check).

Indexes:

- `{ status: 1, createdAt: 1 }` — admin moderation queue, FIFO
- `{ targetType: 1, targetId: 1 }` — “reports against this item” lookup (used for suspicious-behavior heuristics)

6.9 notifications — Owner: Engineer 4

Purpose: User-facing alert. P1 for delivery/UI, but the Product Brief master prompt asks the schema to be prepared now since several MVP flows (match created, report resolved) naturally produce notification-worthy events.

Decision: include the collection in the MVP schema, defer the delivery/read UI to P1. Reasoning: it costs nothing to create the collection and write events into it as a side effect of match/handover/report actions already happening; deferring the *UI* (bell icon, mark-as-read screen) keeps scope disciplined without losing the data.

Field	Type	Required	Default	Notes
<code>_id</code>	ObjectId	generated	—	

Field	Type	Required	Default	Notes
<code>recipientId</code>	ObjectId → <code>users</code>	✓	—	
<code>type</code>	String enum	✓	—	<code>match_created</code> , <code>match_accepted</code> , <code>report_resolved</code> , <code>org_verification_decided</code>
<code>title</code>	String	✓	—	
<code>message</code>	String	✓	—	
<code>relatedEntity</code>	Object { <code>type</code> , <code>id</code> }	optional	<code>null</code>	bounded polymorphic pointer, same pattern as <code>reports.targetType/targetId</code>
<code>readAt</code>	Date	optional	<code>null</code>	
<code>createdAt</code>	Date	auto	—	no <code>updatedAt</code> needed — notifications are write-once

Indexes: { `recipientId`: 1, `readAt`: 1, `createdAt`: -1 } — unread-first inbox query.

6.10 contributions — Owner: Engineer 4

Purpose: An append-only ledger of impact-relevant events (a completed transfer, a fulfilled request), used to derive contribution history (FR-018) and admin/impact metrics (Section 26 of the brief) — **without** inventing environmental conversion figures, per the brief’s explicit instruction.

Field	Type	Required	Default	Notes
<code>_id</code>	ObjectId	generated	—	
<code>type</code>	String enum	✓	—	<code>transfer_completed</code> (MVP only; room for future types)
<code>handoverId</code>	ObjectId → <code>handovers</code>	✓	—	
<code>providerId</code>	ObjectId → <code>users</code>	✓	—	
<code>seekerId</code>	ObjectId → <code>users</code>	✓	—	
<code>categoryId</code>	ObjectId → <code>categories</code>	✓	—	denormalized for category-level impact reporting
<code>quantity</code>	Number	✓	—	denormalized from the resource at completion time
<code>createdAt</code>	Date	auto	—	write-once, no <code>updatedAt</code>

Dynamic vs. stored reputation/impact — the decision: Reputation and contribution `counts` are **calculated from `contributions` at read time for the profile/impact pages** (accurate, no drift, and the collection is small enough at MVP scale that aggregation is cheap), but **cached as denormalized counters on `users`** (`stats.completedTransfers`, `reputationScore`) for hot paths like match-ranking or list badges, updated by the same service call that writes the `contributions` document. This avoids two failure modes: (a) purely dynamic aggregation being too slow if reused on every match score calculation, and (b) purely stored counters drifting from the source of truth with no way to audit them. `contributions` is always the source of truth; `users.stats` is a cache that can be rebuilt from it.

Indexes: { `providerId`: 1, `createdAt`: -1 }, { `seekerId`: 1, `createdAt`: -1 }, { `categoryId`: 1, `createdAt`: -1 } (admin category-impact reporting).

07 — Embedding vs. Referencing Decisions

The Product Brief already states the default direction: **reference, don't deep-embed** (Section 18/19 of the brief). This section justifies every deviation and every non-trivial reference individually, per the required six-question analysis.

07.1 `resource.location` and `resource.availabilityWindow` — Embed

1. Why embed? Always read together with the resource; never queried as a standalone entity; small and fixed-shape.
2. Why not reference? There is no independent “Location” concept in this product — it’s an attribute, not an entity.
3. Read frequency: very high (every browse/list query).
4. Update frequency: low (edited occasionally by the provider).
5. Unbounded growth? No — fixed two-field object.
6. Duplication/consistency risk? None — it only ever exists on the one resource. Decision: **embed**.

07.2 `resources.categoryId` / `requests.categoryId` — Reference

1. Why reference? Categories are shared, admin-managed, reused across thousands of resources/requests.
2. Why not embed? Embedding category name/description into every resource would mean an admin renaming a category requires rewriting every resource document — a classic embedding anti-pattern.
3. Read frequency: high, but a category has ~7–10 values total — trivial to `populate()` or `$lookup` cheaply.
4. Update frequency of the category itself: rare.
5. Unbounded growth: n/a (few categories).
6. Consistency: embedding would create stale category names across historical resources. Decision: **reference** — matches the Product Brief directly (“Referenced by Resource, Request”).

07.3 `organizations.ownerUserId`, `resources.providerId`, `requests.requesterId` — Reference

1. Why reference? These are ownership/authorization pointers, not display data — the backend needs the ID for permission checks far more often than it needs the full user document.
2. Why not embed the user? A user’s profile changes independently of any single resource; embedding would require fan-out updates across every listing when a user edits their name.
3. Read frequency: every authorization check (very high) — but only the ID is needed, not the whole document, so a reference is actually *cheaper* than an embed here.
4. Update frequency: user profile edits are independent of resource edits.
5. Unbounded growth: a user can own many resources — embedding resources into the user document would create an unbounded array (explicitly prohibited).
6. Consistency: referencing avoids needing to propagate profile edits everywhere. Decision: **reference**, and this is also *why* `resources` / `requests` are their own top-level collections rather than embedded arrays inside `users`.

07.4 `matches.providerId` / `matches.requesterId` — Reference + deliberate denormalization

1. Why reference the canonical value? `users` is still the source of truth.
2. Why also copy it onto `matches`? The single most common query in the product (“show me my matches”) needs to filter by provider/requester directly. Without the copy, every match-inbox query would require a two-hop join (`matches` → `resources/requests` → `providerId/requesterId`).

3. Read frequency: very high (match inbox is a core screen).
4. Update frequency: a resource's `providerId` never changes after creation (ownership isn't transferable in this MVP) — so the copied value cannot go stale.
5. Unbounded growth: n/a, scalar field.
6. Consistency risk: effectively zero, because the source field is immutable. This is the one place in the schema where a small, deliberate duplication is safe and worth the query-simplicity gain — the opposite of blind embedding, this is a *justified, narrow* denormalization. Decision: **reference** `resourceId / requestId` as the canonical link, plus a denormalized copy of `providerId / requesterId` for query performance.

07.5 matches.scoreBreakdown — Embed

1. Why embed? It's small (5 numeric fields), always read with the match, never queried independently.
2. Why not reference? There's no separate "Score" entity — it's intrinsic to the match. 3–6. Same reasoning pattern as 07.1. Decision: **embed**.

07.6 handovers.matchId (1:1 with matches) — Reference

1. Why reference instead of embedding the handover fields directly onto `matches`? A `match` can exist and be rejected/expired without ever reaching handover — embedding handover fields onto every match would mean most matches carry unused, always-null handover fields. Keeping `handovers` separate keeps `matches` focused on scoring/proposal and `handovers` focused on confirmation, and lets Engineer 4 own the handover lifecycle without touching Engineer 3's collection.
2. Read/update frequency: handover confirmation is updated twice (once per side) — isolating it avoids write contention on the `matches` document. Decision: **reference**, one `handover` per accepted `match`, enforced by a unique index on `matchId`.

07.7 reports.targetId (User → Report, Resource → Report) — Polymorphic reference

Already justified in Section 6.8. Not a true "embed vs reference" question — it's a *reference shape* question, resolved with `targetType + targetId` rather than three nullable foreign keys, which is the standard safe MongoDB pattern for one-of-several parent types.

07.8 organizations → verification — Embed

1. Why embed? Verification state is intrinsic to the organization, always read with it, and has a small bounded shape (status, documents array, reviewer, timestamps, reason).
2. Why not a separate `verifications` collection? There is exactly one verification record per organization in the MVP (no multi-round history requirement stated in the brief) — a separate collection would just be a 1:1 join with no benefit. Decision: **embed**, with `submittedDocuments` explicitly bounded (see Section 13) to avoid the unbounded-array anti-pattern.

07.9 users.stats (reputation cache) — Embed, with contributions as source of truth

Already justified in Section 6.10. Embed the cache (small, fixed-shape, high-read), keep the audit trail (`contributions`) as its own collection (append-only, potentially large over time, independently queryable by category/date).

08 — Schema Specifications (Mongoose)

Field-level specs, presented the way the brief's master prompt requests: type, required, default, enum, reference, validation, example. Full Mongoose code lives in each engineer's task steps (Sections 20–23); this section is the specification those schemas must satisfy.

8.1 User

SCHEMA

11 lines

```
_id          ObjectId  generated
name         String   required, trim, minlength 2, maxlength 80
email        String   required, unique, lowercase, trim, match: email regex
passwordHash String   required, select: false // never returned by default queries
role         String   required, enum: ['user','admin'], default: 'user'
location     { city: String, area: String }
status       String   required, enum: ['active','suspended'], default: 'active'
emailVerifiedAt Date
reputationScore Number required, default: 0, min: 0
stats        { completedTransfers: Number default 0, failedTransfers: Number default 0 }
timestamps: true
```

Example document:

JSON

7 lines

```
{
  "name": "Mona Tarek",
  "email": "mona@example.com",
  "role": "user",
  "location": { "city": "Cairo", "area": "Maadi" },
  "status": "active"
}
```

8.2 Organization

SCHEMA

13 lines

```
_id          ObjectId  generated
ownerUserId  ObjectId  required, ref: 'User'
name         String   required, trim, minlength 2, maxlength 120
description  String   maxlength 1000
contactInfo  { phone: String, email: String, address: String }
verification {
  status:      String   required, enum:['pending','approved','rejected','suspended'], default:'pending'
  submittedDocuments: [String] validate: maxlength(array) ≤ 10
  reviewedBy:  ObjectId ref: 'User'
  reviewedAt:  Date
  rejectionReason: String
}
timestamps: true
```

8.3 Category

SCHEMA

6 lines

```
_id      ObjectId generated
name     String   required, unique, trim
slug     String   required, unique, lowercase
description String
isActive Boolean   required, default: true
timestamps: true
```

8.4 Resource

SCHEMA

14 lines

```
_id      ObjectId generated
providerId ObjectId required, ref: 'User'
providerOrgId ObjectId ref: 'Organization'
categoryId ObjectId required, ref: 'Category'
title     String   required, minlength 3, maxlength 100
description String   required, maxlength 1000
quantity  Number   required, min: 1
location  { city: String required, area: String }
availabilityWindow { start: Date required, end: Date required }
status    String   required, enum: ['draft','published','available','matched',
                                'accepted','in_handover','completed','impact_recorded',
                                'expired','cancelled','unavailable'], default: 'draft'

safetyDisclosure String
timestamps: true
```

Custom validator: `availabilityWindow.end > availabilityWindow.start`.

8.5 Request

SCHEMA

11 lines

```
_id      ObjectId generated
requesterId ObjectId required, ref: 'User'
requesterOrgId ObjectId ref: 'Organization'
categoryId ObjectId required, ref: 'Category'
quantity  Number   required, min: 1
urgency   String   required, enum: ['low','medium','high'], default: 'medium'
location  { city: String required, area: String }
description String   maxlength 500
status    String   required, enum: ['draft','published','matched','accepted',
                                'fulfilled','cancelled','expired'], default: 'draft'

timestamps: true
```

8.6 Match

SCHEMA

12 lines

```
_id            ObjectId generated
resourceId     ObjectId required, ref: 'Resource'
requestId      ObjectId required, ref: 'Request'
providerId     ObjectId required, ref: 'User'    // denormalized, see 07.4
requesterId    ObjectId required, ref: 'User'    // denormalized, see 07.4
score          Number   required, min: 0, max: 1
scoreBreakdown { category: Number, location: Number, quantity: Number,
                    urgency: Number, availability: Number }
status         String   required, enum: ['proposed','accepted','rejected','expired'],
                        default: 'proposed'
expiresAt      Date
timestamps: true
```

8.7 Handover

SCHEMA

14 lines

```
_id            ObjectId generated
matchId        ObjectId required, unique, ref: 'Match'
resourceId     ObjectId required, ref: 'Resource'
requestId      ObjectId required, ref: 'Request'
providerId     ObjectId required, ref: 'User'
seekerId       ObjectId required, ref: 'User'
confirmedByProvider Boolean required, default: false
confirmedBySeeker Boolean required, default: false
providerConfirmedAt Date
seekerConfirmedAt Date
status         String   required, enum: ['in_progress','completed','cancelled','no_show'],
                        default: 'in_progress'
completedAt    Date
timestamps: true
```

8.8 Report

SCHEMA

10 lines

```
_id            ObjectId generated
reporterId     ObjectId required, ref: 'User'
targetType     String   required, enum: ['resource','request','user']
targetId       ObjectId required
reason         String   required, enum: ['spam','fraud','inappropriate','safety','other']
description     String   maxLength 500
status         String   required, enum: ['open','reviewed','resolved'], default: 'open'
reviewedBy     ObjectId ref: 'User'
resolution     String
timestamps: true
```

8.9 Notification

SCHEMA

9 lines

```
_id          ObjectId generated
recipientId  ObjectId required, ref: 'User'
type         String   required, enum: ['match_created', 'match_accepted',
                                'report_resolved', 'org_verification_decided']
title        String   required
message      String   required
relatedEntity { type: String, id: ObjectId }
readAt       Date
createdAt: auto (no updatedAt)
```

8.10 Contribution

SCHEMA

8 lines

```
_id          ObjectId generated
type         String   required, enum: ['transfer_completed']
handoverId    ObjectId required, ref: 'Handover'
providerId    ObjectId required, ref: 'User'
seekerId      ObjectId required, ref: 'User'
categoryId    ObjectId required, ref: 'Category'
quantity      Number   required, min: 1
createdAt: auto (no updatedAt)
```

Mongoose notes: use `timestamps: true` on every collection except the two append-only ledgers (`notifications`, `contributions`), which only need `createdAt`. No plugins are recommended beyond `mongoose` itself and `bcrypt` for password hashing — anything more (e.g. `mongoose-unique-validator`) is unnecessary since Mongoose + a unique index already produces a catchable duplicate-key error.

09 — Enum & Lifecycle Design

9.1 users.role

Value	Meaning	Who sets it
<code>user</code>	default individual account	system, at registration
<code>admin</code>	platform operator	manually provisioned/seeded, never self-assigned via API

9.2 resources.status — Resource Lifecycle

Matches the Product Brief's Section 11 exactly, plus the edge states it names.

State Transition Matrix

Current State	Action	Next State	Actor
<code>draft</code>	publish	<code>published</code>	provider
<code>published</code>	system marks open	<code>available</code>	system
<code>available</code>	match proposed	<code>matched</code>	system (matching service)
<code>matched</code>	both sides accept	<code>accepted</code>	provider + requester
<code>matched</code>	requester rejects	<code>available</code>	requester (match discarded, resource re-enters pool)
<code>accepted</code>	handover starts	<code>in_handover</code>	system
<code>in_handover</code>	both confirm	<code>completed</code>	provider + seeker
<code>completed</code>	impact logged	<code>impact_recorded</code>	system
<code>published</code> / <code>available</code>	availability window ends	<code>expired</code>	system (scheduled job)
any non-terminal	provider cancels	<code>cancelled</code>	provider
<code>published</code> / <code>available</code> / <code>matched</code>	provider marks unavailable	<code>unavailable</code>	provider
<code>matched</code>	no-show / timeout	<code>available</code>	system (match released, per Product Brief)

Explicitly invalid transitions (rejected by the backend, not just discouraged):

- `completed` → `available` — a completed transfer cannot un-complete.
- `impact_recorded` → `*` — terminal.
- `cancelled` → `*` — terminal; provider must create a new listing.

Enforcement: a single service function `transitionResourceStatus(resource, action, actor)` holds the transition table as data (not scattered `if` statements) and looks up whether `(currentStatus, action)` is allowed before writing. This keeps the rule in one place and testable.

9.3 requests.status — Request Lifecycle

Current State	Action	Next State	Actor
draft	publish	published	requester
published	match proposed + accepted	matched → accepted	system / both sides
accepted	handover completes	fulfilled	system
published / matched	requester cancels	cancelled	requester
published	no match within window (optional)	expired	system (scheduled job, MVP-optional)

Only states justified by the product are included — no separate “in_review” state, since the brief does not describe request-level pre-moderation beyond the general report/flag mechanism.

9.4 matches.status

Value	Meaning
proposed	system generated, awaiting accept/reject from provider and/or requester
accepted	both sides agreed to proceed → triggers <code>resources.status = accepted</code> , creates <code>handovers</code> doc
rejected	either side rejected → resource returns to <code>available</code> , request stays <code>published</code>
expired	timeout with no response → resource returns to <code>available</code>

9.5 handovers.status

`in_progress` → `completed` (both confirmations true) | `cancelled` | `no_show`. Both `cancelled` and `no_show` release the underlying `resources` / `requests` back toward `available` / `published` via the same transition service used in 9.2/9.3.

9.6 organizations.verification.status

`pending` → `approved` | `rejected`; `approved` → `suspended` (policy violation, admin-only). Only an admin may write to this field — enforced at the authorization layer, documented here at the schema layer.

9.7 reports.status

`open` → `reviewed` → `resolved`. `reviewedBy` and `resolution` become required once status leaves `open` (Mongoose custom validator, not a schema-level “required if” since Mongoose doesn’t support conditional required declaratively across two fields cleanly for beginners — do it in a `pre('save')` hook, explained in Engineer 4’s tasks).

10 — Index Strategy

Every index below is tied to a named query need — no speculative indexes.

Collection	Index	Supports	Compound?	Notes
users	{ email: 1 } unique	login, duplicate prevention	no	
users	{ status: 1 }	admin account list	no	low cardinality, still useful combined with pagination sort
organizations	{ ownerId: 1 }	“my organization”	no	
organizations	{ "verification.status": 1, createdAt: 1 }	admin verification queue	yes	status first (equality filter), createdAt second (sort) — correct ESR order
categories	{ slug: 1 } unique	lookup by slug	no	
categories	{ isActive: 1 }	filter dropdowns	no	tiny collection, index mostly for query plan clarity
resources	{ status: 1, categoryId: 1, createdAt: -1 }	browse by category	yes	equality (status, categoryId) then sort (createdAt) — ESR rule
resources	{ providerId: 1, status: 1 }	“my resources”	yes	
resources	{ "location.city": 1, "location.area": 1, status: 1 }	location filter	yes	
resources	{ "availabilityWindow.end": 1 }	expiry job	no	
requests	{ status: 1, categoryId: 1, urgency: -1, createdAt: -1 }	matching candidate retrieval	yes	most selective/most-used filters first
requests	{ requesterId: 1, status: 1 }	“my requests”	yes	
requests	{ "location.city": 1, "location.area": 1, status: 1 }	location filter	yes	
matches	{ resourceId: 1, status: 1 }	matches for a resource	yes	
matches	{ requestId: 1, status: 1 }	matches for a request	yes	
matches	{ providerId: 1, status: 1, createdAt: -1 }	provider match inbox	yes	
matches	{ requesterId: 1, status: 1, createdAt: -1 }	requester match inbox	yes	
matches	{ resourceId: 1, requestId: 1 } unique, partial (status:"proposed")	duplicate-proposal prevention	yes	partial index keeps it cheap — only active proposals indexed
handovers	{ matchId: 1 } unique	1:1 enforcement	no	
handovers	{ providerId: 1, status: 1 }, { seekerId: 1, status: 1 }	pending handovers	yes	two separate indexes, not one combined — providers and seekers query independently
reports	{ status: 1, createdAt: 1 }	moderation queue, FIFO	yes	

Collection	Index	Supports	Compound?	Notes
reports	{ targetType: 1, targetId: 1 }	reports against an item	yes	
notifications	{ recipientId: 1, readAt: 1, createdAt: -1 }	unread inbox	yes	
contributions	{ providerId: 1, createdAt: -1 }, { seekerId: 1, createdAt: -1 }, { categoryId: 1, createdAt: -1 }	contribution history + category impact	yes	

Discussed and explicitly rejected:

- A text index on `resources.title` / `description` for full-text search — the brief’s FR-022 asks for category/location filtering, not free-text search; adding it now would be an unrequested feature.
- Indexing every field “just in case” — write overhead scales with index count; each index here maps to a stated query.
- A single mega-compound index across unrelated filters (e.g. status+category+location+urgency all in one) — MongoDB can only efficiently use a compound index for a prefix of its fields in order, so indexes are scoped to real query field combinations rather than maximized speculatively.

Compound index ordering (ESR rule) is applied throughout: **E**quality fields first, then **S**ort fields, then **R**ange fields — explained plainly to the team in Engineer 3’s onboarding task (Section 22) since the matching-candidate query is the most performance-sensitive one in the system.

11 — Validation Strategy

Three layers, matching the Product Brief's Section 21 exactly:

1. **Frontend (Angular Reactive Forms)** — immediate feedback, mirrors backend rules, does **not** get trusted by the database layer.
2. **Backend (Express + Mongoose schema validation)** — the actual enforcement point. Every write endpoint validates against the Mongoose schema before touching business logic (NFR-009).
3. **MongoDB collection-level validation (`$jsonSchema`)** — used sparingly, only as a last-resort safety net for the handful of fields where an application-layer bug could otherwise corrupt data no later check would catch (e.g. `handovers.status` enum, `resources.quantity` non-negative). Most validation lives in Mongoose because it produces far better error messages for beginners to debug; raw MongoDB schema validation is added only where the team wants defense-in-depth against a direct/incorrect driver write bypassing Mongoose.

Why backend validation is mandatory even with frontend validation: the frontend can be bypassed entirely — direct API calls (Postman, curl), a modified client, or a malicious actor calling the API directly. Only server-side validation is trustworthy (directly from Product Brief Section 21).

12 — Data Integrity Strategy

12.1 Required references

Every reference field (`providerId`, `categoryId`, etc.) is `required` in its schema. The service layer additionally verifies the referenced document actually exists before saving (a missing/garbage `ObjectId` should fail with a clear 400, not silently create an orphaned reference — MongoDB does not enforce foreign keys).

12.2 Valid state transitions

Enforced centrally per Section 9's transition tables — one lookup table per lifecycle, not scattered conditionals.

12.3 Ownership rules

`providerId` / `requesterId` / `reporterId` / `ownerUserId` fields exist specifically so the authorization middleware can compare `req.user._id` against the document's owner field before allowing update/delete. **A user must not be able to update another user's Resource** (Product Brief example) — this is primarily an authorization concern, but the schema makes it checkable.

12.4 Duplicate prevention

- `users.email` unique index.
- `categories.name` / `slug` unique index.
- `matches` partial unique index on (`resourceId`, `requestId`) for active proposals (Section 10).
- `handovers.matchId` unique index (one handover per match).

12.5 Soft delete vs. hard delete

Entity	Strategy	Reasoning
<code>resources</code>	Status-based archival (<code>cancelled</code> / <code>unavailable</code> / <code>expired</code>)	Preserves history for any completed matches/handovers referencing it
<code>requests</code>	Status-based archival (<code>cancelled</code> / <code>expired</code>)	Same reasoning
<code>users</code>	Status-based (<code>suspended</code>), never hard-deleted	Referenced by <code>resources</code> / <code>requests</code> / <code>matches</code> / <code>handovers</code> / <code>contributions</code> — hard delete would orphan history
<code>organizations</code>	Status-based (<code>suspended</code>)	Same reasoning
<code>reports</code>	Never deleted	Moderation history must be auditable
<code>matches</code> / <code>handovers</code> / <code>contributions</code>	Never deleted	These <i>are</i> the historical record the product's trust model depends on
<code>categories</code>	<code>isActive: false</code>	Historical <code>resources</code> / <code>requests</code> keep referencing the category name
<code>notifications</code>	Hard delete allowed (P1, optional)	Purely transient UX data, not part of the trust/impact record

No entity that participates in the trust/impact/handover history is ever hard-deleted. This directly follows the Product Brief's requirement for historical tracking of completed transfers and impact.

13 — Security Strategy

- **Passwords:** hashed with bcrypt before the `User` document is saved; `passwordHash` field has `select: false` so it is never returned by default `find()` queries, only explicitly opted into for the login check.
- **Least privilege:** the app's MongoDB connection user has read/write access to the Dawwarha database only, not admin/cluster privileges.
- **Environment variables:** `MONGODB_URI`, JWT secret, and any third-party keys live in `.env`, never in source control. A `.env.example` file with placeholder keys (no real values) is committed instead.
- **Injection prevention:** Mongoose parameterizes queries by default; the team must avoid building raw `$where` queries or interpolating unsanitized user input into query objects (e.g. never do `User.find({ email: req.body.email })` without first validating `req.body.email` is actually a string — a classic NoSQL-injection vector where an attacker sends an object like `{ "$ne": null }` instead of a string).
- **PII minimization:** `organizations.verification.submittedDocuments` stores references/URLs, not raw uploaded file bytes, in the database (files themselves belong in file storage, out of MongoDB scope for this MVP). `contactInfo` is only surfaced to the other party after match acceptance, per the Product Brief's privacy note — enforced in the API response layer, not the schema, but documented here so engineers don't leak it in an early "browse resources" endpoint.
- **Sensitive field handling:** no field in this schema stores a national ID, payment info, or other high-sensitivity data — deliberately, since the MVP has no such requirement.

14 — Aggregation Strategy

Aggregation is used only where a single query cannot answer the need — not by default.

Pipeline	Purpose	Input	Stages (summary)	Result
Admin dashboard summary	Section 25 metrics (Total Users, Resources Published, Completed Transfers, Open Reports, etc.)	users, resources, requests, handovers, reports	Parallel \$count / \$group queries (can run as separate simple queries — a \$facet pipeline is optional, not required, for the MVP scale)	Dashboard numbers
Resources published vs. fulfilled by category	“undersupplied categories” insight (brief Section 25)	resources grouped by categoryId + status, joined conceptually with fulfilled requests	\$group by categoryId, \$count by status	Table for admin
Category-level impact	Impact Metrics (brief Section 26)	contributions	\$group by categoryId, \$sum: quantity, \$count	Cumulative completed transfers per category
Match acceptance rate	“matching quality” admin decision input	matches	\$group by status, \$count	proposed vs accepted vs rejected ratio

Aggregation is **not** used for: simple “my resources” or “my matches” lists (plain `find()` with indexes is correct and simpler), or anything a single `$group / $count` on one collection can already answer without `$lookup` across collections.

15 — Transaction Strategy

Where a transaction may be appropriate: when a `match` is accepted, three things should happen together — `matches.status` → `accepted`, `resources.status` → `accepted`, and a new `handovers` document created. If the process crashes between step 1 and step 3, the system would be left with an accepted match but no handover record.

Is it actually necessary for this MVP? Judged case by case:

- **Match acceptance (multi-document write):** Yes — this is the one place a MongoDB multi-document transaction is justified, since three collections change together and the accepted/completed state must never exist half-written. Use `session.withTransaction()` wrapping the three writes.
- **Handover confirmation (two fields on one document):** No — this is a single-document update (`confirmedByProvider` / `confirmedBySeeker` on one `handovers` doc), and MongoDB guarantees single-document atomicity natively. A transaction here would be unnecessary overhead.
- **Report creation:** No — single document insert.
- **Contribution + user stats cache update:** Recommended but not strictly required — a small transaction wrapping the `contributions` insert and the `users.stats` counter increment keeps the cache from drifting if one write fails; acceptable to implement without a transaction for MVP speed, with a documented note that the cache can be rebuilt from `contributions` if it ever drifts (see 6.10).

What could go wrong without a transaction where one is used? A partially-applied match acceptance (resource stuck in `matched` while the match itself shows `accepted`) — exactly the kind of inconsistency the brief’s trust model can’t tolerate.

Where transactions should NOT be used: any single-document update, and any read-only aggregation. Overusing transactions adds latency and complexity the MVP doesn’t need.

16 — Seed Data Strategy

Seed data must support the brief’s 5–7 minute demo narrative (Section 37): **Problem → Resource → Request → Match → Handover → Impact → Admin.**

Minimum seed set:

- 2 categories used in the demo (e.g. “Food”, “Furniture”) + 3–5 additional categories for realism
- 1 pre-seeded **provider** user (already has a track record — a few historical **contributions**)
- 1 pre-seeded **seeker/organization** (one verified organization, to demonstrate org flows)
- 1 admin user
- 3–5 additional “background” users/resources/requests so the browse/search screens don’t look empty during the demo, but not seeded exhaustively
- One resource and one request specifically shaped to score highly on the matching formula (same category, same city/area, compatible quantity, high urgency) so the live-demo match is guaranteed to surface
- One already- **completed** handover with a **contributions** record, so the admin dashboard and impact page have non-zero numbers *before* the live demo transfer completes

Separation of concerns: a **seed/** script (Node script, not a UI feature) run manually against a **development** database only — never pointed at a shared/demo database automatically, and never run against production. **NODE_ENV** is checked at the top of the seed script and it refuses to run if **NODE_ENV** `=== 'production'`.

Development seed data vs. production data: development seed uses obviously fake names/emails (e.g. `demo.provider@dawwarha.test`); nothing about the seed script is reused for real onboarding.

17 — API-to-Database Mapping

Endpoint	Collection(s)	Operation	Owner (DB)	Validation	Authorization
POST /api/auth/register	users	insert	Eng 1	email format+unique, password strength	public
POST /api/auth/login	users	find + compare hash	Eng 1	credentials required	public
GET /api/users/me	users	findById	Eng 1	—	authenticated
PUT /api/users/me	users	updateOne	Eng 1	profile field rules	self only
GET /api/resources	resources	find (filtered, paginated)	Eng 2	query params sanitized	public
POST /api/resources	resources	insert	Eng 2	full schema validation	authenticated
PUT /api/resources/:id	resources	updateOne	Eng 2	schema + transition rules	owner only
DELETE /api/resources/:id	resources	status update (archival, not physical delete)	Eng 2	—	owner or admin
GET /api/requests	requests	find (filtered, paginated)	Eng 3	—	authenticated
POST /api/requests	requests	insert	Eng 3	full schema validation	authenticated
PUT/DELETE /api/requests/:id	requests	update / status archival	Eng 3	schema + transition rules	owner or admin
GET /api/matches	matches	find	Eng 3	—	authenticated (own matches)
POST /api/matches/:resourceId/generate	matches, reads requests, resources	insert (scoring service reads candidates)	Eng 3	score inputs validated	resource owner or system job
PUT /api/matches/:id/accept	matches, resources, handovers (transaction)	update + insert	Eng 3 writes match/resource fields it owns is out of scope — actually Eng 4 owns handovers , so this endpoint is a cross-domain write coordinated in the service layer, not the schema layer	transition rules	provider or requester party to the match
PUT /api/matches/:id/reject	matches, resources	update	Eng 3	transition rules	party to the match

Endpoint	Collection(s)	Operation	Owner (DB)	Validation	Authorization
POST <code>/api/transactions/:matchId/confirm</code>	handovers	update (one side's confirmation)	Eng 4	side-specific	provider or seeker of that handover
POST <code>/api/organizations</code>	organizations	insert	Eng 1	schema validation	authenticated
POST <code>/api/organizations/:id/verify</code>	organizations	update <code>verification.status</code>	Eng 1	reason required if rejected	admin only
POST <code>/api/reports</code>	reports	insert	Eng 4	polymorphic target check	authenticated
PUT <code>/api/reports/:id/resolve</code>	reports	update	Eng 4	resolution required	admin only
GET/POST/PUT/DELETE <code>/api/categories</code>	categories	CRUD	Eng 2	name/slug unique	read: public; write: admin
GET <code>/api/admin/analytics</code>	multiple (aggregation, Section 14)	aggregate/read-only	shared, coordinated by Eng 4	—	admin only

Note on cross-domain writes (match acceptance): this is the one endpoint in the system that legitimately touches three engineers' collections in one transaction. It is called out explicitly in Section 25 (Integration Plan) as requiring joint review between Engineer 3 (owns `matches`) and Engineer 4 (owns `handovers`), coordinated through the Shared Database Contract rather than either engineer unilaterally changing the other's schema.

18 — Shared Database Contract

Every engineer follows this without exception.

Naming

- Collections: lowercase, plural (`users`, `resources`).
- Fields: `camelCase`. No mixing with `snake_case`.
- Booleans: prefixed to read as a question where natural (`isActive`, `confirmedByProvider`).

IDs

- All primary keys are MongoDB's default `ObjectId`. No custom string IDs unless a justified reason is documented in this file (none currently).
- All reference fields are named `<entity>Id` (`providerId`, `categoryId`), never `<entity>_id` or bare `<entity>`.

Timestamps

- Every mutable collection uses Mongoose's `timestamps: true` (`createdAt`, `updatedAt`).
- Append-only ledgers (`notifications`, `contributions`) use `createdAt` only.

Enums

- Always lowercase snake-free single words or `snake_case` for multi-word values (`in_handover`, `impact_recorded`) — chosen for readability over camelCase in enum *values* specifically (fields stay camelCase; enum *values* are the one place `snake_case` is allowed, for consistency with typical REST/status-string conventions).
- Every enum's allowed values are defined **once**, in the owning engineer's schema file — never redefined or duplicated in another engineer's code.

Error handling

- Mongoose validation errors are caught by the centralized Express error middleware (per Product Brief Section 22) and returned as `{ success: false, message, errors? }` — the database layer throws normal Mongoose `ValidationError` / `CastError`; it does not format HTTP responses itself.

References

- A reference field always uses `ref: 'ModelName'` in the Mongoose schema (enabling `.populate()`), even on fields that are usually queried by raw ID.

Status values

- Every lifecycle status lives in the transition tables in Section 9. No engineer invents a new status value without updating this document first.

Pagination

- Skip/limit for the MVP (see Section 27), consistent request shape across all list endpoints: `?page=1&limit=20`.

Soft deletion

- Governed by Section 12.5's table. No collection outside that table implements a `deletedAt` field independently.

Validation

- Backend (Mongoose) is the enforcement layer. Frontend validation mirrors it but is never trusted alone.

Index naming

- MongoDB auto-names compound indexes; engineers do not need to hand-name them, but must document each index's purpose in a comment above the `schema.index(...)` call.

Environment variables

- `MONGODB_URI`, `MONGODB_URI_TEST`, `JWT_SECRET` — defined once in `.env.example`, never duplicated with different names by different engineers.

Model export conventions

- Every model file exports a single Mongoose model as `module.exports = mongoose.model('modelName', schema)`, filed at `src/models/<modelName>.js`, matching the file structure in Section 19.

19 – Team Architecture

File structure

SCHEMA

19 lines

```

src/
├─ config/
│   └─ database.js          # connection setup, all engineers use this, owned by Eng 1
├─ models/
│   ├── User.js             # Eng 1
│   ├── Organization.js     # Eng 1
│   ├── Category.js         # Eng 2
│   ├── Resource.js         # Eng 2
│   ├── Request.js          # Eng 3
│   ├── Match.js            # Eng 3
│   ├── Handover.js         # Eng 4
│   ├── Report.js           # Eng 4
│   ├── Notification.js     # Eng 4
│   └─ Contribution.js      # Eng 4
├─ validators/              # per-model input validation helpers (mirrors schema rules)
├─ services/                # business logic: matching engine, lifecycle transitions, verification workflow
├─ repositories/            # thin data-access wrappers around Mongoose models (optional but recommended for
testability)
├─ utils/                   # shared helpers (pagination, ObjectId checks)
└─ seed/                    # seed scripts, Section 16

```

Why this structure: `models/` holds only schema definitions (no business logic) so ownership boundaries map 1:1 to files; `services/` holds the transition tables and matching formula so lifecycle rules live in one place per domain, testable independent of Express routes; `repositories/` is optional for the MVP but recommended because it lets engineers write unit tests against a mockable data layer instead of a real MongoDB connection.

Ownership map

Already stated in Sections 01 and 05 — repeated here as the canonical single source: each `models/*.js` file's git history should show commits from its owning engineer only, with cross-domain changes going through PR review from the owning engineer (Section 24/25).

20 — Engineer 1 Plan — رحمة محمد محمد أحمد

Mission: Own Identity, Users, Organizations & Access — the foundation every other domain depends on for `providerId` / `requesterId` / `ownerUserId` references and role-based authorization support.

Scope: `users`, `organizations` collections; database connection config (`src/config/database.js`), since it's needed before anyone else can start).

Collections Owned: `users`, `organizations`

Prerequisites: Node.js + MongoDB installed locally; access to the shared `.env.example`; read Section 18 (Shared Database Contract) before writing any code.

TASK 1.1 — Database Connection Setup

Objective: Establish the MongoDB connection every other engineer's models will use.

WHY

Nothing else in the project can run without this — it's the literal first dependency in the implementation order (Section 26, Phase 0).

Prerequisites: MongoDB running locally or an Atlas connection string.

Steps:

1. Create `src/config/database.js`.
2. Read `MONGODB_URI` from `process.env` (never hardcode it).
3. Write a `connectDB()` function using `mongoose.connect(uri)`.
4. Log a clear success message on connect.
5. Log a clear error message and `process.exit(1)` on failure (fail loudly, don't limp along without a DB).
6. Add `.env.example` at the project root with `MONGODB_URI=mongodb://localhost:27017/dawwarha_dev` (placeholder, not a real credential).
7. Add `.env` to `.gitignore` if it isn't already.
8. Call `connectDB()` from the app's entry point (`server.js` / `app.js`) before the Express server starts listening.
9. Test: start the app with MongoDB running — confirm the success log appears.
10. Test: stop MongoDB, start the app — confirm the error log appears and the process exits instead of silently continuing.

Expected Result: The app refuses to serve requests without a working database connection, and every other engineer can `require('../config/database')` in their own model tests.

COMMON MISTAKES

hardcoding the connection string; not handling the connection error case; forgetting `await`/promise handling around `mongoose.connect`.

Verification: run the app twice — once with MongoDB up, once with it down — and confirm the two different logged outcomes.

TASK 1.2 — User Model

Objective: Build the `users` collection per Section 8.1.

WHY

Every other collection references users via `providerId/requesterId/etc.` — this must exist first.

Prerequisites: Task 1.1 complete.

Steps:

1. Create `src/models/User.js`.
2. Define the schema fields exactly per Section 8.1 (name, email, passwordHash, role, location, status, emailVerifiedAt, stats).
3. Mark `email` `required`, `unique`, `lowercase`.
4. Mark `passwordHash` `required`, `select: false`.
5. Add `role` `enum` `['user', 'admin']`, default `'user'`.
6. Add `status` `enum` `['active', 'suspended']`, default `'active'`.
7. Add `timestamps: true` in schema options.
8. Create the unique index on `email` (Mongoose does this automatically from `unique: true`, but explicitly run `User.syncIndexes()` once in dev to confirm it's built).
9. Write a small `hashPassword` helper (bcrypt) used by the auth service — **not** inside the schema as a pre-save hook that re-hashes on every save (a common beginner bug: hash only when the password field is actually being set/changed).
10. Create seed data: one admin user, one demo provider, one demo seeker (coordinate with Task 1.6/seed integration).
11. Test: insert a valid user — confirm it saves.
12. Test: insert a duplicate email — confirm MongoDB returns a duplicate-key error, not a silent overwrite.
13. Test: insert a user missing `email` — confirm Mongoose validation rejects it.
14. Test: confirm a plain `User.find()` does **not** return `passwordHash` (because of `select: false`).

Expected Result: A working `User` model enforcing uniqueness, required fields, and password-hash privacy.

COMMON MISTAKES

storing plaintext passwords; forgetting `select: false` (leaking hashes in API responses); re-hashing an already-hashed password on every profile update.

Verification: run the four tests above against a local test database; show a teammate the `passwordHash` field is absent from a normal `GET /api/users/me` response.

TASK 1.3 — Organization Model

Objective: Build the `organizations` collection per Section 8.2.

WHY

Organizations need admin verification before they can publish standing requests (FR-016/FR-017) — this is the persistence layer for that gate.

Prerequisites: Task 1.2 complete (references `User`).

Steps:

1. Create `src/models/Organization.js`.
2. Define fields per Section 8.2, including the nested `verification` object.
3. Set `ownerUserId` as `required`, `ref: 'User'`.
4. Set `verification.status` enum `['pending', 'approved', 'rejected', 'suspended']`, default `'pending'`.
5. Add a `pre('save')` hook or service-layer check: if `verification.status === 'rejected'`, require `verification.rejectionReason` to be non-empty.
6. Bound `verification.submittedDocuments` to a maximum array length (e.g. 10) via a custom validator — prevents an unbounded array.
7. Add indexes per Section 6.2 (`ownerUserId`, and `{ "verification.status": 1, createdAt: 1 }`).
8. Create seed data: one pre-approved organization for the demo.
9. Test: insert a valid pending organization.
10. Test: attempt to set `status: 'rejected'` with no `rejectionReason` — confirm rejection.
11. Test: query the verification queue index (`find({ "verification.status": "pending" }).sort({ createdAt: 1 })`) and confirm it uses the index (`.explain()`).
12. Test: confirm `ownerUserId` referencing a non-existent user is caught at the service layer (write a small existence-check function, since Mongoose itself won't enforce this).

Expected Result: A working `Organization` model with an enforceable verification gate.

COMMON MISTAKES

forgetting the bounded-array validator on `submittedDocuments`; allowing `rejected` without a reason; letting any authenticated user (not just admins) write to `verification.status` (this must be blocked in the route/authorization layer, but the schema/task should note it clearly for whoever builds that route).

Verification: `.explain("executionStats")` on the pending-queue query to confirm an index is used, not a collection scan.

Files To Create: `src/config/database.js`, `src/models/User.js`, `src/models/Organization.js`, `.env.example`, seed fragments for users/organizations. **Files To Modify:** app entry point (to call `connectDB()`), `.gitignore`. **Deliverables:** working, tested `User` and `Organization` models; database connection utility; documented indexes. **Definition of Done:** see Section 29 (applies to every engineer identically). **Dependencies On Other Engineers:** none — Engineer 1 is the foundation (Phase 1 in Section 26). **Handoff Requirements:** `User` and `Organization` models merged to `develop` before Engineers 2–4 build any model that references them (Section 26, Phase 1 → Phase 2 gate).

21 – Engineer 2 Plan – ادهم محمد حسن عبد السميع

Mission: Own the Resource Supply Domain — everything a provider needs to list, categorize, and manage surplus.

Scope: `resources`, `categories` collections.

Collections Owned: `resources`, `categories`

Prerequisites: Engineer 1’s `User` model merged (Phase 1 complete).

TASK 2.1 – Category Model

Objective: Build the controlled taxonomy per Section 8.3.

WHY

Both `resources` and `requests` need a shared, admin-managed category list before either can be built against real data (FR-024).

Steps:

1. Create `src/models/Category.js`.
2. Fields: `name` (required, unique), `slug` (required, unique, lowercase, auto-generated from `name` in a `pre('validate')` hook), `description`, `isActive` (default `true`).
3. Add unique indexes on `name` and `slug`.
4. Seed the 7 categories named in the Product Brief: food, books, furniture, clothing, school supplies, office equipment, other.
5. Test: insert all 7 seed categories — confirm uniqueness works (duplicate `name` rejected).
6. Test: deactivate a category (`isActive: false`) and confirm it’s excluded from an `isActive: true` query but still resolvable by ID (for historical resources).

Expected Result: A small, stable, admin-only-writable taxonomy collection.

COMMON MISTAKES

allowing category deletion instead of deactivation (breaks historical references); building a hierarchical/parent-category structure the MVP doesn’t need.

Verification: confirm a resource referencing a deactivated category still displays correctly (category name still resolvable via `populate()`).

TASK 2.2 – Resource Model + Lifecycle

Objective: Build the `resources` collection per Section 8.4, and the lifecycle transition function per Section 9.2.

WHY

This is the core “supply” entity the entire matching loop depends on.

Prerequisites: Task 2.1 complete, Engineer 1’s `User` model available.

Steps:

1. Create `src/models/Resource.js` with all fields from Section 8.4.
2. Add the custom validator: `availabilityWindow.end > availabilityWindow.start`.
3. Add the `status` enum with all 11 values from Section 9.2, default `'draft'`.
4. Create `src/services/resourceLifecycleService.js` holding the state transition table from Section 9.2 as a plain JS object (`{ draft: { publish: 'published' }, published: { ... }, ... }`), plus a `transitionResource(resource, action, actor)` function that looks up the table and throws a clear error on an invalid transition.
5. Add all four indexes from Section 6.4 / Section 10.
6. Add ownership check helper: `isOwner(resource, userId)` for the authorization layer to use.
7. Create seed data: several resources across categories/statuses, including at least one shaped to score well against the demo request (Section 16).
8. Test: create a resource with valid data — confirm it saves in `draft` status.
9. Test: attempt `availabilityWindow.end` before `start` — confirm rejection.
10. Test: call `transitionResource` through the full happy path (`draft → published → available → matched → accepted → in_handover → completed → impact_recorded`) — confirm each step succeeds.
11. Test: attempt `transitionResource(resource, 'reopen')` from `completed` — confirm it's rejected as invalid.
12. Test: confirm the `{status, categoryId, createdAt}` index is used by `.explain()` on a typical browse query.

Expected Result: A working `Resource` model plus a reusable, tested lifecycle service other tasks (matching, handover) will call into.

COMMON MISTAKES

implementing lifecycle checks as scattered `if` statements across multiple route handlers instead of one central table-driven function; forgetting the partial/compound index ordering (equality fields before sort fields).

Verification: unit test file exercising every transition in the table, both valid and invalid.

Files To Create: `src/models/Category.js`, `src/models/Resource.js`, `src/services/resourceLifecycleService.js`, seed fragments. **Deliverables:** tested `Category` / `Resource` models, tested lifecycle service, documented indexes. **Dependencies On Other Engineers:** `User` model (Engineer 1). **Handoff Requirements:** `Resource` / `Category` merged before Engineer 3 can build `matches` (Phase 2 → Phase 3 gate, Section 26). Engineer 3 consumes `resourceLifecycleService` for match-acceptance transitions rather than re-implementing resource state logic.

22 — Engineer 3 Plan — دينا عبدالله عليان محمد

Mission: Own Community Needs & Matching — requests, and the explainable scoring engine that connects them to resources.

Scope: `requests`, `matches` collections.

Collections Owned: `requests`, `matches`

Prerequisites: Engineer 1's `User` model and Engineer 2's `Resource` / `Category` models merged.

TASK 3.1 — Request Model + Lifecycle

Objective: Build `requests` per Section 8.5 and its transition table per Section 9.3.

Steps:

1. Create `src/models/Request.js` with fields from Section 8.5.
2. Add `urgency` enum `['low', 'medium', 'high']`, default `'medium'`.
3. Add `status` enum per Section 9.3, default `'draft'`.
4. Create `src/services/requestLifecycleService.js` mirroring the pattern from Task 2.2 (table-driven transitions).
5. Add indexes from Section 6.5.
6. Seed data: several requests across categories/urgency, including the demo-matched request (Section 16).
7. Test: create/validate a request; test invalid urgency value rejected; test the ESR-ordered compound index with `.explain()`.

Expected Result: working, tested `Request` model and lifecycle service, structurally parallel to Engineer 2's `Resource` / lifecycle pair.

COMMON MISTAKES

duplicating Engineer 2's transition-table pattern incorrectly (copy the *pattern*, not resource-specific states); forgetting `requesterOrgId` is optional (only organizations with standing requests use it).

Verification: unit tests for every transition, valid and invalid.

TASK 3.2 — Matching Engine Data Model

Objective: Build `matches` per Section 8.6 and the scoring persistence logic.

WHY

This is the product's flagship feature (Product Brief Section 12) — must be explainable, not a black box.

Prerequisites: Task 3.1, Engineer 2's `Resource` model.

Steps:

1. Create `src/models/Match.js` with fields from Section 8.6, including the embedded `scoreBreakdown` object.

2. Add the partial unique index on `(resourceId, requestId)` filtered to `status: "proposed"` (Section 10) — look up MongoDB's `partialFilterExpression` syntax; this is a new concept for beginners, so document it inline with a comment explaining *why* (prevents duplicate active proposals for the same pair).
3. Create `src/services/matchingService.js`:
 - `calculateScore(resource, request)` — implements `category×W1 + location×W2 + quantity×W3 + urgency×W4 + availability×W5`, each sub-score normalized 0–1, weights as named constants at the top of the file (not magic numbers).
 - `generateMatches(resourceId)` — queries candidate `requests` (status `published`, same category, reasonable location) using the Section 10 indexes, scores each, filters by a minimum threshold constant, and inserts `Match` documents.
4. **Explicitly do not** call any external service, embedding model, or ML library here — this is pure arithmetic over normalized fields (per the brief's explicit “do not overengineer matching” instruction).
5. Add the remaining indexes from Section 6.6.
6. Test: `calculateScore` unit tests with hand-computed expected values for a few input combinations (category match/mismatch, same/different location, etc.).
7. Test: `generateMatches` against seeded data — confirm the demo resource/request pair scores above threshold and produces a `Match` document.
8. Test: attempt to insert two simultaneous `proposed` matches for the same `(resourceId, requestId)` — confirm the partial unique index rejects the second.
9. Test: match accept/reject flow updates `matches.status` correctly (coordinate with Engineer 4 for the handover-creation side — see Task 4.1).

Expected Result: A working, unit-tested, explainable matching engine whose score breakdown can be displayed to both sides of a match.

COMMON MISTAKES

hardcoding weights inline instead of named constants (makes tuning during testing painful); computing a score without storing the breakdown (defeats the “explainable” requirement); accidentally making `generateMatches` an unbounded operation with no threshold/limit (could produce a huge number of low-quality matches).

Verification: print a `scoreBreakdown` for a real seeded pair and manually verify the arithmetic by hand.

Files To Create: `src/models/Request.js`, `src/models/Match.js`, `src/services/requestLifecycleService.js`, `src/services/matchingService.js`. **Deliverables:** tested `Request` / `Match` models, tested matching service with documented, tunable weight constants. **Dependencies On Other Engineers:** `User`, `Resource`, `Category` (Engineers 1–2). **Handoff Requirements:** `matchingService.generateMatches()` and the match-accept transition are consumed by Engineer 4's handover-creation logic (Phase 3 → Phase 4 gate). Coordinate the match-acceptance transaction (Section 15) jointly — Engineer 3 updates `matches` / `resources` status, Engineer 4's `handovers` document is created in the same transaction.

23 – Engineer 4 Plan – عبدالستار محمد عبدالستار مطاولع

Mission: Own Transfer, Trust, Moderation & Impact — the domain that turns an accepted match into a verified, recorded outcome, and keeps the platform trustworthy.

Scope: `handovers`, `reports`, `notifications`, `contributions` collections; cross-domain integration testing (per the Product Brief’s team structure, which names this role as also owning end-to-end UX/integration consistency).

Collections Owned: `handovers`, `reports`, `notifications`, `contributions`

Prerequisites: Engineer 3’s `Match` model merged.

TASK 4.1 – Handover Model + Two-Sided Confirmation

Objective: Build `handovers` per Section 8.7 and the confirmation state machine per Section 6.7.1.

Steps:

1. Create `src/models/Handover.js` with fields from Section 8.7.
2. Add unique index on `matchId`.
3. Create `src/services/handoverService.js`:
 - `createHandoverForMatch(match)` — called as part of the match-acceptance transaction (coordinate with Engineer 3, Section 15).
 - `confirm(handoverId, side, userId)` — `side` is `'provider'` or `'seeker'`; verifies `userId` matches the corresponding owner field, sets the confirmation flag idempotently, and if both flags are now true, sets `status: 'completed'`, `completedAt: now()`, and calls into `contributionService.recordCompletedTransfer(handover)` (Task 4.4).
4. Handle `cancel` / `no_show` transitions, releasing the underlying resource/request back toward `available` / `published` by calling Engineer 2’s/3’s lifecycle services (do not duplicate their transition tables here — call them).
5. Test: full happy path — create handover, confirm provider side, confirm seeker side, verify `status` becomes `completed` and `contributions` gets a document.
6. Test: confirm the same side twice — verify idempotency (no error, no duplicate timestamp overwrite issue).
7. Test: confirm as a user who is neither the provider nor seeker on that handover — verify rejection.
8. Test: cancel a handover mid-flow — verify the resource returns to `available`.

Expected Result: A working two-sided confirmation flow that structurally cannot mark a transfer complete from one side alone.

COMMON MISTAKES

letting either party set both flags; forgetting idempotency (a double-click causing an error); not cascading the cancel/no-show transition back into resources/requests.

Verification: attempt a one-sided “complete” via direct document manipulation in a test and confirm the *service* layer, not just the UI, prevents it — the point of FR-013 is a backend guarantee.

TASK 4.2 – Report Model (Polymorphic)

Objective: Build `reports` per Section 8.8.

Steps:

1. Create `src/models/Report.js` with `targetType` / `targetId` polymorphic fields.
2. Add a service-layer function `validateReportTarget(targetType, targetId)` that checks the ID actually exists in the right collection (`resources`, `requests`, or `users`) before allowing the report to save — since MongoDB cannot enforce this at the schema level.
3. Add the `pre('save')` -or-service-layer rule: `reviewedBy` / `resolution` required once `status` leaves `open`.
4. Add indexes from Section 6.8.
5. Test: report each of the three target types successfully.
6. Test: report a `targetId` that doesn't exist for the given `targetType` — verify rejection.
7. Test: resolve a report without a resolution string — verify rejection.

Expected Result: Working polymorphic reporting with real target validation, not just a schema pass.

COMMON MISTAKES

trusting `targetId` without checking it resolves; using three optional foreign-key fields instead of the polymorphic pair.

TASK 4.3 — Notification Model

Objective: Build `notifications` per Section 8.9 (schema now, minimal write-side integration; delivery UI is explicitly P1).

Steps:

1. Create `src/models/Notification.js`.
2. Add a small `notificationService.notify(recipientId, type, title, message, relatedEntity)` helper.
3. Call it (optionally, as a stretch task, not blocking) from `matchingService.generateMatches` (`match_created`) and `reportService` (`report_resolved`) — coordinate with Engineers 3 on the call site without modifying Engineer 3's files directly (Engineer 3 calls Engineer 4's exported `notificationService`, not the reverse).
4. Test: insert a notification, confirm `readAt` defaults to `null`, confirm the unread-inbox index (`.explain()`).

Expected Result: A working notification write path; UI/read-marking flow explicitly deferred to P1.

TASK 4.4 — Contribution Ledger + Reputation Cache

Objective: Build `contributions` per Section 8.10 and the cache-update logic described in Section 6.10.

Steps:

1. Create `src/models/Contribution.js`.
2. Create `src/services/contributionService.js`:
 - `recordCompletedTransfer(handover)` — inserts a `Contribution` document, then increments `users.stats.completedTransfers` for both `providerId` and `seekerId` (and recomputes `reputationScore` using a simple, documented formula — e.g. `completedTransfers / (completedTransfers + failedTransfers)`, clamped, or a simpler count-based score for MVP; document whichever is chosen).
 - `getContributionHistory(userId)` — reads from `contributions`, the source of truth, for the profile/impact page (FR-018).
 - `rebuildStatsCache(userId)` — a maintenance function that recomputes `users.stats` by aggregating `contributions`, in case the cache ever drifts (documented as the fix for any observed inconsistency, per Section 6.10's design decision).
3. Add indexes from Section 6.10.

4. Test: complete a handover end-to-end (via Task 4.1) and confirm a `Contribution` document is created and `users.stats.completedTransfers` increments for both parties.
5. Test: `rebuildStatsCache` against seeded data produces the same numbers as the incremental updates.

Expected Result: A working, auditable impact ledger with a rebuildable cache — no fabricated environmental/monetary figures anywhere in this model, per the Product Brief’s explicit instruction.

COMMON MISTAKES

inventing an “impact score” formula not grounded in the brief (e.g. a CO2-savings estimate) — explicitly prohibited; storing only the cache without the ledger (loses auditability).

Files To Create: `src/models/Handover.js`, `Report.js`, `Notification.js`, `Contribution.js`, `src/services/handoverService.js`, `reportService.js`, `notificationService.js`, `contributionService.js`. **Deliverables:** all four collections, tested; the two-sided confirmation guarantee; the polymorphic report/notification pattern documented for reuse; the ledger+cache impact model. **Dependencies On Other Engineers:** `Match` (Engineer 3), `Resource` / `Request` lifecycle services (Engineers 2–3, for cancel/no-show cascades). **Handoff Requirements:** `handoverService.confirm` is the function the API’s `/api/transactions/:matchId/confirm` endpoint calls directly — no other engineer re-implements confirmation logic.

24 — Git & Collaboration Strategy

Branching: `main` (stable) ← `develop` (integration) ← `feature/database/<domain>` branches, one per engineer:

- `feature/database/identity` (Engineer 1)
- `feature/database/resources` (Engineer 2)
- `feature/database/matching` (Engineer 3)
- `feature/database/trust-impact` (Engineer 4)

This nests inside the Product Brief's own stated project-wide strategy (trunk-based, short-lived feature branches, PR-only merges to `main`, Section 34) — the database team uses `develop` as an integration branch before the wider team's `main`.

Commit naming: `db(<domain>): <what changed>` — e.g. `db(resources): add availabilityWindow validator`.

Pull Requests: required for every merge into `develop`; minimum one reviewer, and any PR touching another engineer's model file requires that engineer's explicit approval, not just any teammate's.

Merge order: follows the Implementation Order in Section 26 — Engineer 1's PR merges first (nothing else can be tested without it), then Engineer 2, then Engineer 3, then Engineer 4.

Conflict resolution: because ownership is 1:1 per collection, file-level conflicts should be rare; if two engineers need to touch a shared file (e.g. `src/config/database.js`, `src/utils/pagination.js`), that file's changes go through a short sync message in the team channel before opening the PR.

Schema change communication: any change to a field, enum, or index that another engineer's code already depends on (e.g. Engineer 3 changing `matches.status` values) must be posted in the shared channel and reflected in this document (Section 18) before merging — this document is living, not a one-time artifact.

25 — Integration Plan

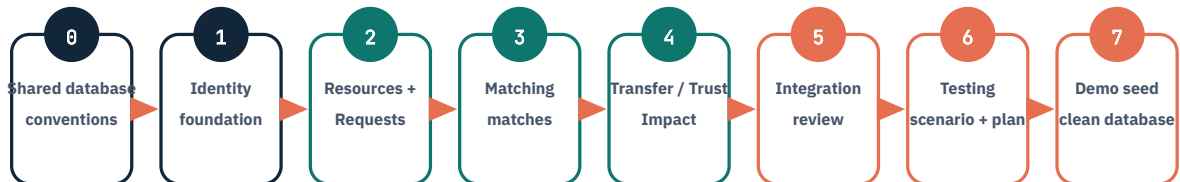
Once all four branches are ready, integration proceeds through:

1. **Schema Review** — every model file reviewed against Section 8’s specifications side by side; flag any field that drifted from spec.
2. **Naming Review** — grep the codebase for `snake_case` field names, `_id` suffix inconsistencies, and non-`camelCase` slips.
3. **Reference Review** — confirm every `ref:` string matches an actual registered Mongoose model name exactly (a common source of silent `populate()` failures).
4. **Index Review** — run `db.collection.getIndexes()` on every collection in a shared dev database; compare against Section 10’s table; remove any accidental duplicate index.
5. **Validation Review** — attempt every “Common Mistake” listed in Sections 20–23 against the merged code, confirm each is actually rejected.
6. **Seed Review** — run the full seed script from a clean database; confirm the demo narrative (Section 16) plays out without manual data patching.
7. **Integration Testing** — run the full End-to-End scenario (Section 27).
8. **API Contract Verification** — walk the API-to-Database Mapping (Section 17) endpoint by endpoint, confirming each is wired to the right collection/operation.
9. **Final Database Audit** — the checklist in Section 28.

The **match-acceptance transaction** (Section 15/17) is called out as the one integration point requiring Engineers 3 and 4 to pair-review together, since it’s the single place their two domains write in the same operation.

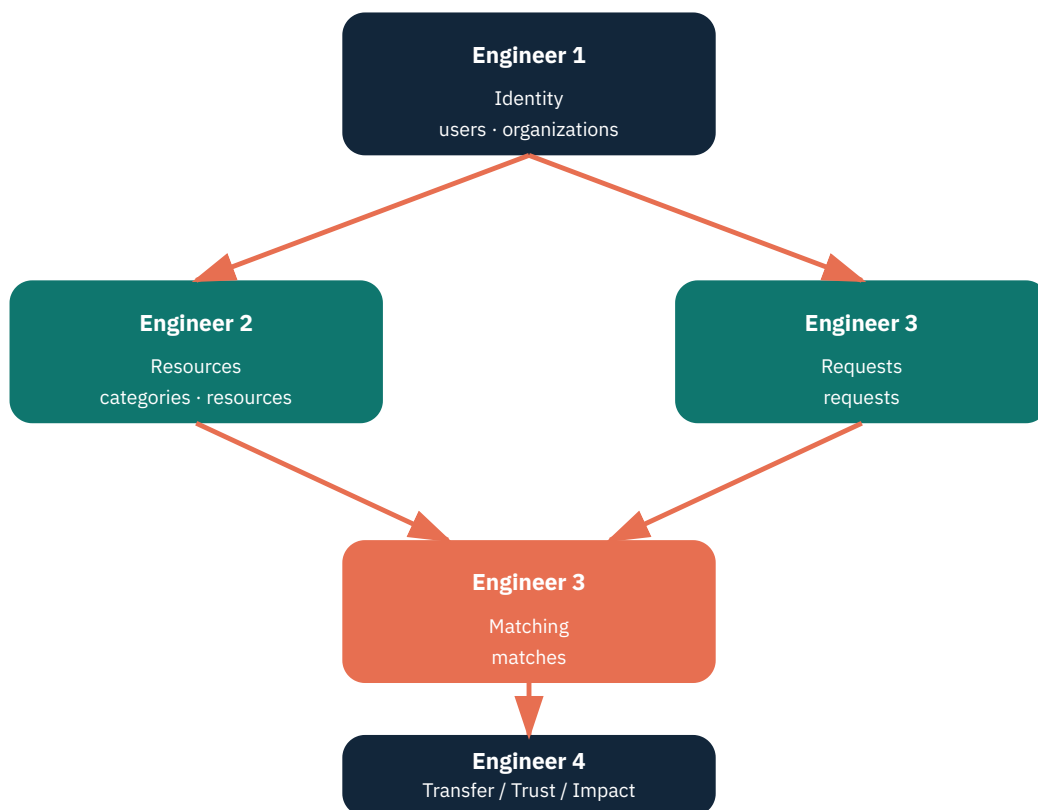
26 — Implementation Order

DELIVERY / IMPLEMENTATION ORDER



Dependency graph:

DELIVERY / DEPENDENCY GRAPH



Engineers 2 and 3 can build `resources` and `requests` in parallel once Engineer 1's `User` model merges — they don't depend on each other until Engineer 3 reaches the `matches` task, which needs both.

27 — Database Test Plan

Connection Tests

- ☐ Database connects with a valid `MONGODB_URI`
- ☐ App fails loudly (not silently) with an invalid/missing `MONGODB_URI`
- ☐ Environment variables load correctly from `.env`

Schema Tests (per collection)

- ☐ Required fields reject missing values
- ☐ Types reject wrong-typed values (e.g. string into `quantity`)
- ☐ Enums reject out-of-range values
- ☐ Defaults apply when a field is omitted
- ☐ `timestamps: true` populates `createdAt` / `updatedAt`

Constraint Tests

- ☐ Duplicate `email` rejected
- ☐ Duplicate `category` name/slug rejected
- ☐ Invalid `ObjectId` references caught at the service layer
- ☐ Invalid lifecycle states rejected by each transition service

CRUD Tests — for every collection: create, read (single + filtered list), update, and (status-based) delete/archive.

Authorization Support Tests — confirm every collection with an ownership field (`providerId`, `requesterId`, `ownerUserId`, `reporterId`) actually exposes that field for the authorization middleware to check.

Lifecycle Tests — every valid transition in Sections 9.2/9.3/9.4/9.5/9.6/9.7 succeeds; every explicitly-invalid transition (Section 9.2's examples, etc.) is rejected.

Matching Tests

- ☐ Category mismatch → low/zero category sub-score
- ☐ Location difference → reduced location sub-score
- ☐ Quantity mismatch → reduced quantity sub-score
- ☐ Urgency correctly weighted
- ☐ Availability window correctly scored
- ☐ Total score persists correctly and matches the sum of the weighted breakdown
- ☐ Below-threshold candidates are not surfaced as matches

Trust Tests

- ☐ Report creation for all three target types
- ☐ Report resolution requires reviewer + resolution
- ☐ Organization verification approve/reject/suspend transitions
- ☐ Two-sided confirmation cannot be completed by one side alone

Impact Tests

- ☐ Completed transfer creates exactly one `Contribution` document
 - ☐ Both confirmations required before `Contribution` is written
 - ☐ `rebuildStatsCache` matches incrementally-updated `users.stats`
-

28 — End-to-End Database Scenario

A single scenario proving all four domains interoperate.

#	Step	Collection Touched	Operation	Important Fields	State Transition	Expected Result
1	Create provider	users	insert	role: 'user'	—	provider account exists
2	Create seeker	users	insert	role: 'user'	—	seeker account exists
3	Create organization	organizations	insert	ownerUserId, verification.status: 'pending'	—	org exists, unverified
4	Admin approves organization	organizations	update	verification.status	pending → approved	org can publish standing requests
5	Create category	categories	insert	name, slug	—	category exists
6	Provider creates resource	resources	insert	providerId, categoryId, quantity	— → draft	resource exists
7	Provider publishes resource	resources	update	status	draft → published → available	resource browsable
8	Seeker creates request	requests	insert	requesterId, categoryId, urgency	— → draft	request exists
9	Seeker publishes request	requests	update	status	draft → published	request open for matching
10	Matching service evaluates candidates	reads resources, requests	query	category/location/quantity/urgency/availability	—	ranked candidate list computed
11	Match is created	matches	insert	resourceId, requestId, score, scoreBreakdown	— → proposed	explainable match exists
12	Match is accepted (transaction)	matches, resources, handovers	update, update, insert	matches.status, resources.status, new handovers doc	proposed → accepted; available → matched → accepted; handover in_progress	coordinated three-document write
13	Provider confirms	handovers	update	confirmedByProvider, providerConfirmedAt	stays in_progress	one-sided confirmation recorded
14	Seeker confirms	handovers	update	confirmedBySeeker, seekerConfirmedAt	in_progress → completed	transfer becomes real

#	Step	Collection Touched	Operation	Important Fields	State Transition	Expected Result
15	Contribution recorded	contributions, users.stats	insert, update	providerId, seekerId, quantity, cached counters	—	impact ledger entry + reputation cache updated
16	Resource marked impact-recorded	resources	update	status	completed → impact_recorded	lifecycle closed
17	Admin dashboard reads aggregate data	users, resources, requests, handovers, reports, contributions	aggregate/read-only	counts and category breakdowns	—	dashboard reflects the just-completed transfer

This scenario is the basis for the team’s integration test suite (Section 25, item 7) and mirrors the Product Brief’s own demo narrative (Section 37 of the brief) almost exactly, which is intentional — the database must support the actual demo, not a hypothetical one.

29 — Definition of Done

A database task is **not** complete because “the model was created.” It is complete only when **all** of the following hold:

- ☐ Schema exists and matches Section 8’s specification exactly (or a documented, reviewed deviation)
 - ☐ Validation exists at the Mongoose layer for every required/constrained field
 - ☐ Every index is justified by a named query in Section 10 (no speculative indexes)
 - ☐ Relationships (embed/reference) match Section 07’s decisions, or a new decision is documented there
 - ☐ CRUD behavior is tested (happy path)
 - ☐ Invalid cases are tested (missing fields, bad enums, bad references, invalid transitions)
 - ☐ Seed data exists where the collection needs demo-visible data
 - ☐ Ownership fields exist and are documented for the authorization layer to consume
 - ☐ Code follows the Shared Database Contract (Section 18) — naming, timestamps, enum style, error handling
 - ☐ Another engineer (not the owner) has reviewed the PR
 - ☐ No blocking integration issue remains open against the collection
-

30 — Student Execution Checklists

Each engineer follows the same checklist shape, applied to their own collections.

Checklist — Engineer 1 (رحمة محمد محمد أحمد)

- ☐ Read the Database Contract (Section 18)
- ☐ Create `feature/database/identity` branch
- ☐ Build `database.js` connection config
- ☐ Create `User` model (fields, enums, defaults, `timestamps`)
- ☐ Add `email` unique index, `passwordHash` `select:false`
- ☐ Create `Organization` model (nested `verification`)
- ☐ Add bounded-array validator on `submittedDocuments`
- ☐ Add both models' indexes
- ☐ Create seed data (admin, demo provider, demo seeker, demo org)
- ☐ Test valid insertion for both models
- ☐ Test invalid insertion (missing fields, bad enum, duplicate email)
- ☐ Test queries against each index with `.explain()`
- ☐ Test organization verification transitions
- ☐ Document both models against Section 8
- ☐ Open Pull Request
- ☐ Request review
- ☐ Fix review comments
- ☐ Merge to `deveLop`

Checklist — Engineer 2 (ادهم محمد حسن عبد السميع)

- ☐ Read the Database Contract
- ☐ Confirm Engineer 1's models are merged
- ☐ Create `feature/database/resources` branch
- ☐ Create `Category` model + seed the 7 taxonomy values
- ☐ Create `Resource` model with full field set
- ☐ Add `availabilityWindow` custom validator
- ☐ Build `resourceLifecycleService.js` (table-driven transitions)
- ☐ Add all `resources / categories` indexes
- ☐ Create seed data (multiple resources, one demo-matched)
- ☐ Test valid/invalid insertion
- ☐ Test every lifecycle transition, valid and invalid
- ☐ Test indexes with `.explain()`
- ☐ Document models against Section 8
- ☐ Open Pull Request, request review, fix comments, merge

Checklist — Engineer 3 (دينا عبدالله عليان محمد)

- ☐ Read the Database Contract
- ☐ Confirm Engineer 1 + Engineer 2 merged
- ☐ Create `feature/database/matching` branch
- ☐ Create `Request` model + `requestLifecycleService.js`
- ☐ Create `Match` model with embedded `scoreBreakdown`
- ☐ Add the partial unique index on `(resourceId, requestId)`
- ☐ Build `matchingService.js` (named weight constants, `calculateScore`, `generateMatches`)
- ☐ Add all `requests` / `matches` indexes
- ☐ Create seed data including the guaranteed demo match
- ☐ Unit test `calculateScore` against hand-computed expected values
- ☐ Test duplicate-proposal prevention
- ☐ Test match accept/reject transitions
- ☐ Document models against Section 8
- ☐ Open Pull Request, request review, fix comments, merge

Checklist — Engineer 4 (عبدالستار محمد عبدالستار مطاوع)

- ☐ Read the Database Contract
- ☐ Confirm Engineer 3 merged
- ☐ Create `feature/database/trust-impact` branch
- ☐ Create `Handover` model + `handoverService.js` (two-sided confirmation, idempotent)
- ☐ Create `Report` model + polymorphic target validation
- ☐ Create `Notification` model + minimal write-side integration
- ☐ Create `Contribution` model + `contributionService.js` (ledger + cache + rebuild function)
- ☐ Add all four collections' indexes
- ☐ Create seed data (one pre-completed handover + contribution for non-zero demo numbers)
- ☐ Test full handover confirmation happy path
- ☐ Test one-sided completion is structurally impossible
- ☐ Test report polymorphic target validation
- ☐ Test contribution ledger + cache consistency
- ☐ Pair-review the match-acceptance transaction with Engineer 3
- ☐ Document models against Section 8
- ☐ Open Pull Request, request review, fix comments, merge

31 — Final Architecture Summary

Complete Collection List: `users`, `organizations`, `categories`, `resources`, `requests`, `matches`, `handovers`, `reports`, `notifications`, `contributions` — 10 collections, each with exactly one owner (Section 05).

Complete Relationship Map: `organizations.ownerUserId → users`; `resources.providerId → users`, `resources.categoryId → categories`; `requests.requesterId → users`, `requests.categoryId → categories`; `matches.resourceId → resources`, `matches.requestId → requests` (+ denormalized `providerId / requesterId → users`); `handovers.matchId → matches (1:1)`; `reports.targetId → resources|requests|users` (polymorphic); `notifications.recipientId → users`; `contributions.handoverId → handovers`, `.providerId/.seekerId → users`, `.categoryId → categories`.

Complete Index Map: Section 10's table — 24 indexes total across 10 collections, every one query-justified.

Complete Enum Map: `users.role`, `users.status`, `organizations.verification.status`, `resources.status` (11 values), `requests.status` (7 values), `matches.status`, `handovers.status`, `reports.targetType`, `reports.reason`, `reports.status`, `notifications.type` — all defined once in Section 9, never duplicated.

Complete Lifecycle Map: Resource lifecycle (9.2), Request lifecycle (9.3), Match lifecycle (9.4), Handover lifecycle (9.5), Organization verification lifecycle (9.6), Report lifecycle (9.7) — six independent, table-driven state machines, one per collection that needs one.

Complete Ownership Map: Section 05's table, restated in Sections 20–23's headers.

Complete API-to-Database Map: Section 17, covering every endpoint in the Product Brief's Section 20 API design.

Complete Testing Map: Section 27's checklist plus Section 28's end-to-end scenario.

Closing note for the team: this document is the single source of truth for the database layer, the same way the Product Brief is the single source of truth for the product. If a task ever seems to require inventing a collection, field, or status not listed here, the correct next step is to update this document first — through the Shared Database Contract process in Section 18/24 — not to improvise silently inside one engineer's branch.